SYSTEM DESIGN DOCUMENT

# AI Secure Coding Assistant

Autonomous n8n workflow for vulnerability detection, secure patch generation, and report automation.

| Document Field | Value |
|---|---|
| Assessment Option | Option 2 - Secure Coding Assistant |
| Automation Platform | n8n |
| LLM Provider | OpenRouter Chat Model |
| RAG Layer | Pinecone Vector Store with Gemini Embeddings |
| Primary Output | Patched source code, unified diff, verification status, Markdown report |

**Executive Summary**

This system automates secure code review. It accepts source files, normalizes and line-numbers code, uses an AI Agent with RAG context to identify vulnerabilities, generates a patched version, checks patch artifacts, and produces a professional report with audit logging support.

## 1. Problem Statement

Manual secure code review is time-consuming, inconsistent, and difficult to scale across many files and languages. The goal is to build an autonomous AI agent that can analyze vulnerable source code, explain security weaknesses, generate secure fixes, and preserve original functionality after patching.

**Goal**

Build an autonomous AI agent capable of identifying software vulnerabilities and automatically applying secure fixes to source code.

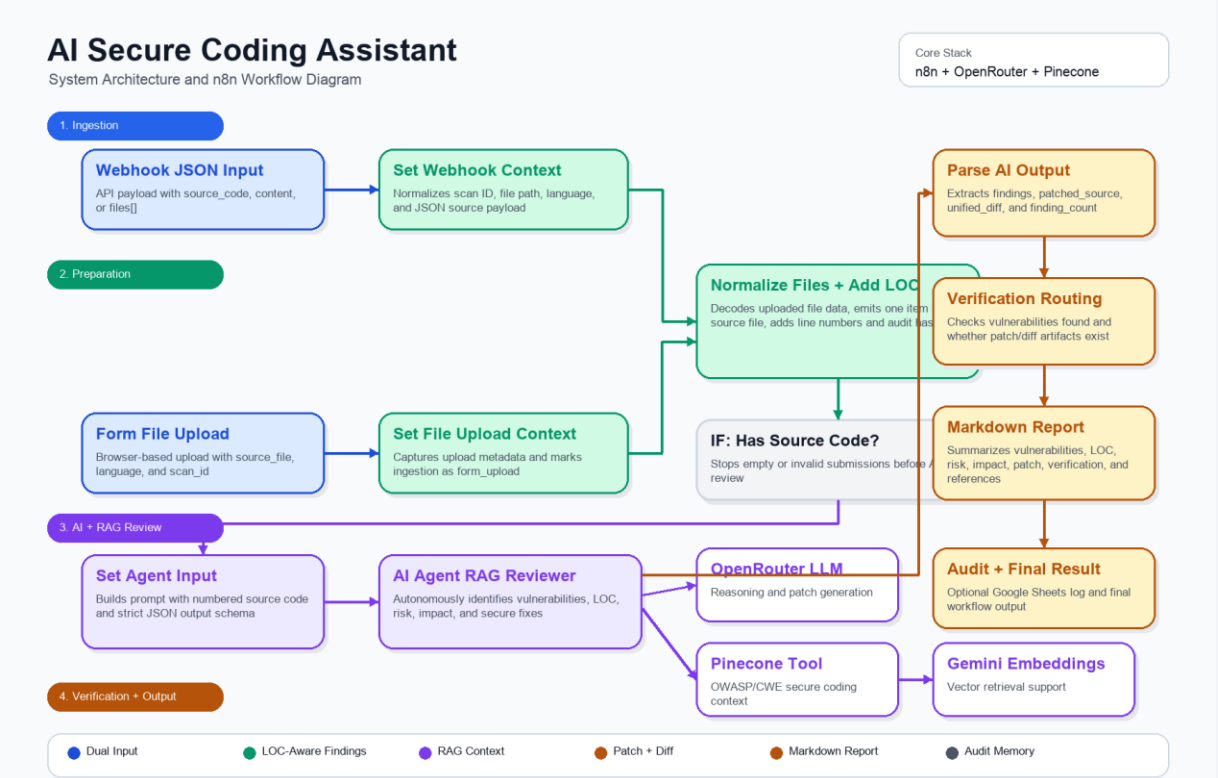
## 2. Requirements

| Requirement | How the Workflow Addresses It |
|---|---|
| Accept vulnerable source code file | Supports JSON/API input through Webhook and browser-based file upload through n8n Form Trigger. |
| Identify vulnerability | AI Agent analyzes numbered source code and returns structured findings. |
| Identify vulnerable LOC | Normalization step adds line numbers before AI analysis. |

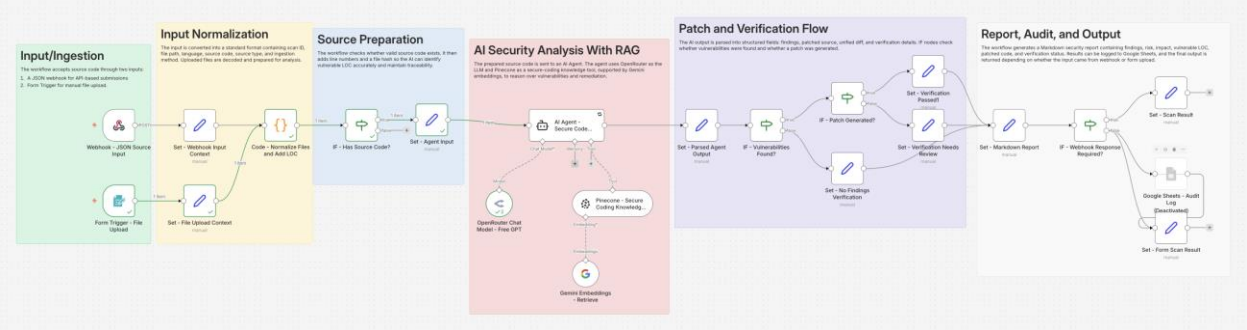
| | |
|---|---|
| Explain risk and impact | Prompt schema requires vulnerability, risk, impact, evidence, and remediation. |
| Generate patched code | AI Agent returns patched_source and unified_diff fields. |
| Preserve original functionality | Prompt requires functionality preservation notes and secure minimal changes. |
| Professional report | Set - Markdown Report node builds a structured Markdown security report. |
| Multiple files | Normalization outputs one item per file, allowing the downstream pipeline to process each file independently. |
| Automatic verification | IF nodes check vulnerability presence and patch/diff generation status. |

## 3. System Workflow Diagram

### Workflow Diagram



## 4. Major Workflow Stages




### 1. Ingestion

The system accepts source code using two entry points: Webhook for API/JSON payloads and Form Trigger for user-friendly file upload demos.

### 2. Normalization

Incoming data is standardized into scan_id, file_path, language_hint, source_type, source_code, line_count, numbered_source, and file_hash.

### 3. AI Review Preparation

The workflow creates a structured agent prompt containing numbered code and a strict JSON response schema.

### 4. RAG-Based AI Analysis

The AI Agent uses OpenRouter for reasoning and Pinecone as a secure-coding knowledge tool, with Gemini embeddings supporting retrieval.

### 5. Patch Verification

IF nodes determine whether vulnerabilities were found and whether patched source and unified diff artifacts were generated.

### 6. Reporting and Audit

The system generates a Markdown report and can log scan history to Google Sheets for auditability.

## 5. Feature Highlights

| Feature | Description | Business/Assessment Value |
|---|---|---|
| Dual input modes | Webhook and form upload | Easy to demo and easy to integrate |

| | | |
|---|---|---|
| | | into automation systems. |
| Autonomous LLM reasoning | AI Agent decides vulnerabilities and fixes | Avoids hard-coded responses and demonstrates agentic behavior. |
| RAG security context | Pinecone retrieval tool supplies secure-coding knowledge | Improves accuracy and explainability. |
| LOC-aware analysis | Source code is line-numbered before review | Meets vulnerable LOC requirement. |
| Patch generation | Returns patched_source and unified_diff | Shows concrete remediation output. |
| Verification routing | IF nodes check findings and patch generation | Demonstrates automated validation logic. |
| Markdown reporting | Report includes findings, risk, impact, patch, and references | Meets professional report bonus. |
| Audit memory | Optional Google Sheets log | Supports long-term traceability and review history. |

## 6. Node-Level Architecture

| Layer | Node | n8n Node Type |
|---|---|---|
| Input | Webhook - JSON Source Input | n8n-nodes-base.webhook |
| Input | Form Trigger - File Upload | n8n-nodes-base.formTrigger |
| Input | Set - Webhook Input Context | n8n-nodes-base.set |
| Logic/Transform | Set - File Upload Context | n8n-nodes-base.set |
| Logic/Transform | Code - Normalize Files and Add LOC | n8n-nodes-base.code |
| Logic/Transform | IF - Has Source Code? | n8n-nodes-base.if |
| AI/RAG | Set - Agent Input | n8n-nodes-base.set |
| AI/RAG | AI Agent - Secure Code RAG Reviewer | @n8n/n8n-nodes-langchain.agent |
| AI/RAG | OpenRouter Chat Model - Free GPT | @n8n/n8n-nodes-langchain.lmChatOpenRouter |
| AI/RAG | Pinecone - Secure Coding Knowledge Tool | @n8n/n8n-nodes-langchain.vectorStorePinecone |
| AI/RAG | Gemini Embeddings - Retrieve | @n8n/n8n-nodes-langchain.embeddingsGoogleGemini |
| AI/RAG | Set - Parsed Agent Output | n8n-nodes-base.set |
| Logic/Transform | IF - Vulnerabilities Found? | n8n-nodes-base.if |
| Logic/Transform | IF - Patch Generated? | n8n-nodes-base.if |

| | | |
|---|---|---|
| Logic/Transform | Set - Verification Needs Review | n8n-nodes-base.set |
| Logic/Transform | Set - No Findings Verification | n8n-nodes-base.set |
| Logic/Transform | Set - Markdown Report | n8n-nodes-base.set |
| Input | IF - Webhook Response Required? | n8n-nodes-base.if |
| Output/Audit | Google Sheets - Audit Log | n8n-nodes-base.googleSheets |
| Logic/Transform | Set - Scan Result | n8n-nodes-base.set |
| Logic/Transform | Set - Form Scan Result | n8n-nodes-base.set |
| Logic/Transform | Set - Verification Passed1 | n8n-nodes-base.set |

## 7. Data Flow

1. User submits source code through the webhook or file-upload form.
2. Input context is standardized by Set nodes.
3. Code node converts source into per-file items and adds line numbers/hash metadata.
4. IF node filters out empty source submissions.
5. Set node builds the AI Agent prompt with a strict JSON schema.
6. AI Agent analyzes code with OpenRouter and retrieves secure-coding guidance from Pinecone.
7. Parsed output is routed through vulnerability and patch verification checks.
8. Markdown report is generated and optionally logged to Google Sheets.
9. Webhook submissions return final JSON; form submissions expose the report in execution output.

## 8. Multi-File and Scalability Design

The normalization step can emit one n8n item per uploaded file or per entry in a JSON files array. Because downstream nodes operate item-by-item, the same review pipeline can scan multiple files in a single execution. For production-grade parallelism, the design can be extended with n8n queue mode, multiple workers, or a child workflow that scans one file per execution.

| Scale Concern | Current Handling | Production Extension |
|---|---|---|
| Multiple files | One item per source file | Dispatch each item to a child scanner workflow. |
| Large codebases | File-level analysis | Chunk files, summarize modules, and use queue workers. |
| Rate limits | Single LLM agent path | Batch files and throttle concurrent AI calls. |
| Verification depth | Checks patch/diff artifacts | Run Semgrep, npm audit, tests, or language syntax checks. |

## 9. Security and Reliability Considerations

- API keys are managed through n8n credentials, not hard-coded in the workflow.

- The workflow produces patch artifacts instead of silently modifying production code.
- Reports include limitations because static workflow checks do not replace complete application testing.
- Google Sheets audit logging can be enabled for traceability.
- Human approval can be added before applying patches to real application codebases.

## 10. Expected Outputs

| Output | Description |
|---|---|
| Findings | Structured vulnerability list with severity, confidence, CWE/OWASP mapping where available, evidence, risk, impact, and remediation. |
| Patched Source | Full corrected source code generated by the AI Agent. |
| Unified Diff | Before/after patch representation for review. |
| Verification Status | Pass or needs-review status based on patch artifact checks. |
| Markdown Report | Professional report suitable for submission or PDF conversion. |